Индексы в PostgreSQL

Виды индексов, применение и примеры

B-Tree индекс

- Наиболее часто используемый индекс
- Для равенства и диапазонных запросов (=, <, >, BETWEEN)
- Покрывает 90% потребностей в запросах

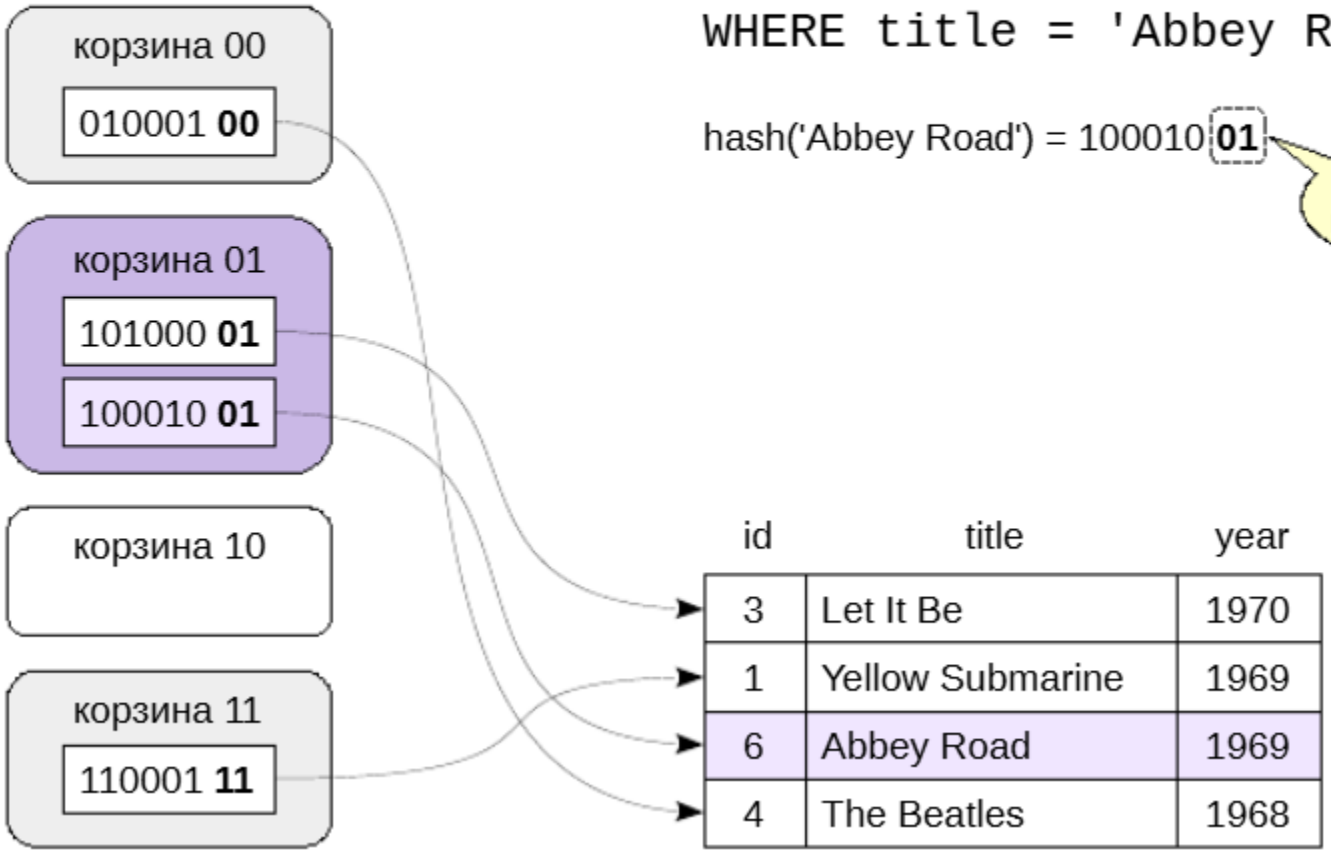
CREATE INDEX idx_name ON table(column);

Hash индекс

```
CREATE INDEX ON t USING hash(title);
SELECT * FROM t
WHERE title = 'Abbey Road';
```

hash('Abbey Road') = 100010**01**

номер корзины



id	title	year
3	Let It Be	1970
1	Yellow Submarine	1969
6	Abbey Road	1969
4	The Beatles	1968

Hash индекс

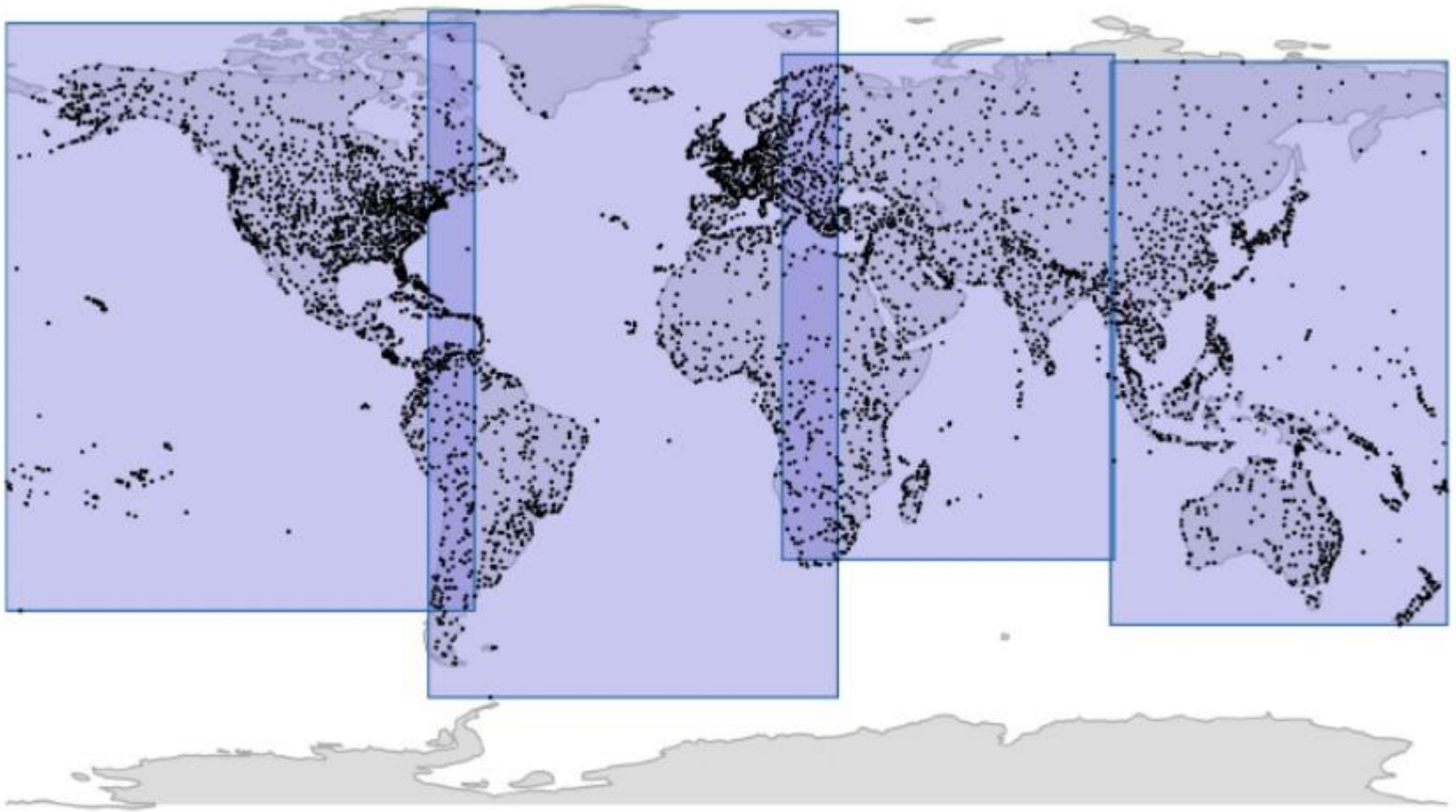
- Хранит только хеш-коды, но не исходные значения
- Размер не зависит от ключа индексирования
- Невозможно сканирование только индекса
- Индекс увеличивается динамически (скачкообразный рост)

В отличие от B-дерева, хеш-индексы имеют много ограничений, в частности:

- единственная поддерживаемая операция — поиск по условию **равенства**, поскольку хеш-функция не сохраняет порядок следования значений;
- не поддерживается ограничение **уникальности**;
- **нельзя** создать **многоколоночный** индекс и добавить к индексу дополнительные include-столбцы.

Хеш-индексы не получили широкого распространения. Однако за счет меньшего размера и фиксированного времени поиска хеш-индекс может в ряде случаев работать быстрее, чем индекс на основе B-дерева

GiST индекс



GiST индекс

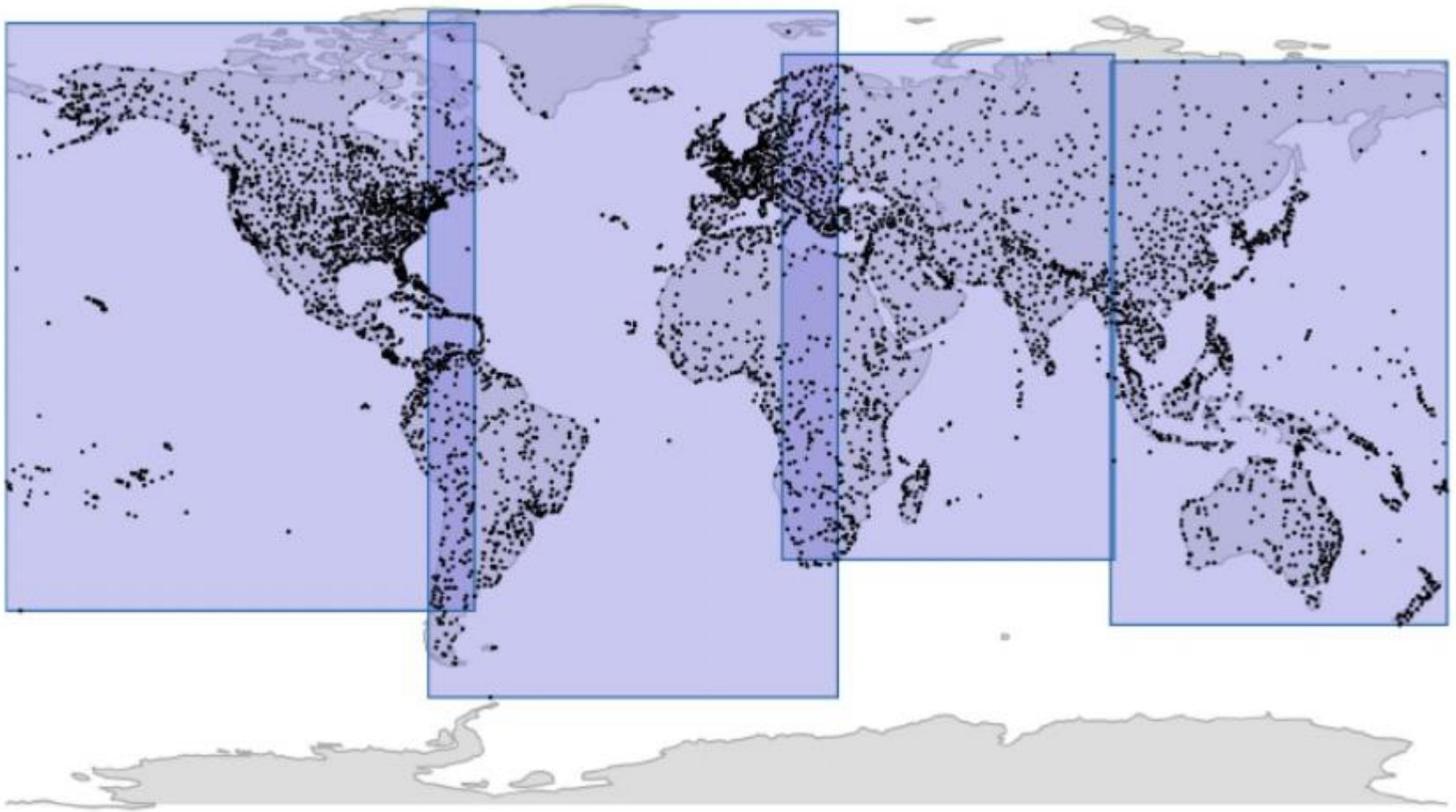
- GiST расшифровывается как **generalized search tree** — обобщенное дерево поиска.
- Поддерживает геометрические объекты, полнотекстовый поиск

CREATE INDEX idx_gist_name ON table USING gist(column);

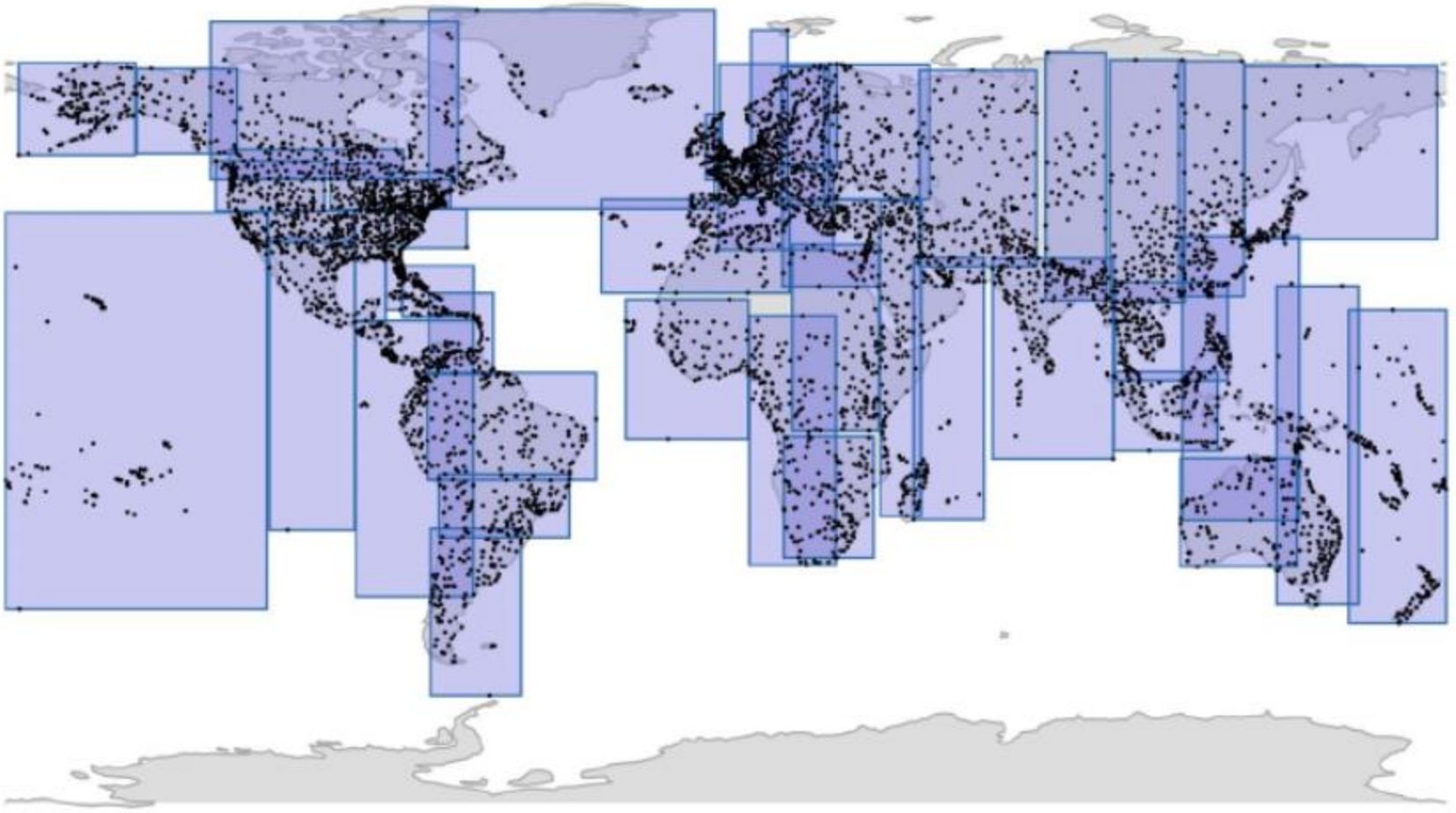
Идею работы GiST-индексов рассмотрим на примере точек на плоскости. Плоскость разбивается на несколько прямоугольников, которые в сумме покрывают все индексируемые точки. Эти прямоугольники составляют верхний уровень дерева. Как видно на рисунке, прямоугольники могут пересекаться (хотя это и уменьшает эффективность поиска).

- …
- Такой алгоритм индексирования называется R-деревом.

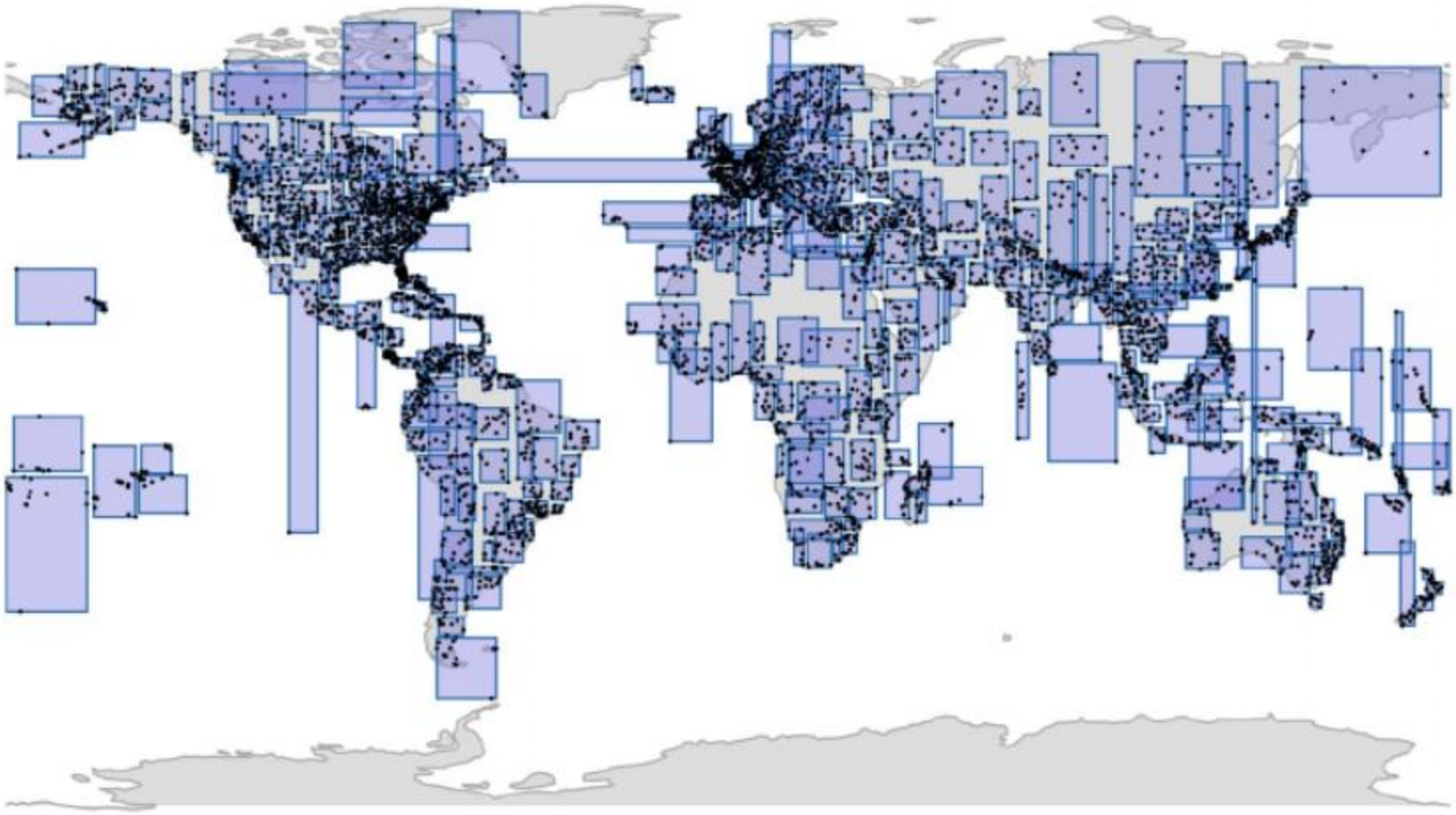
GiST индекс



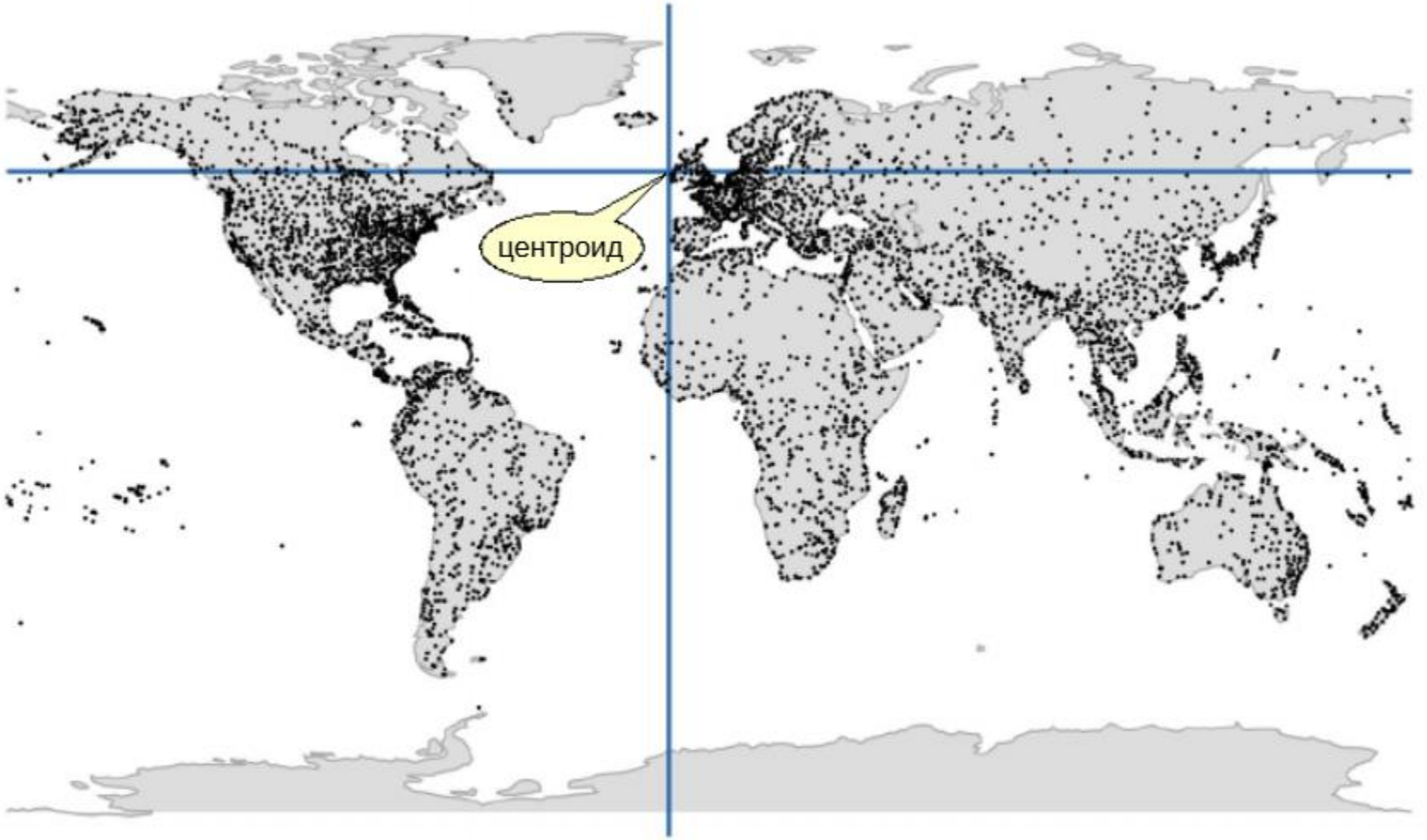
GiST индекс



GiST индекс



SP-GiST индекс



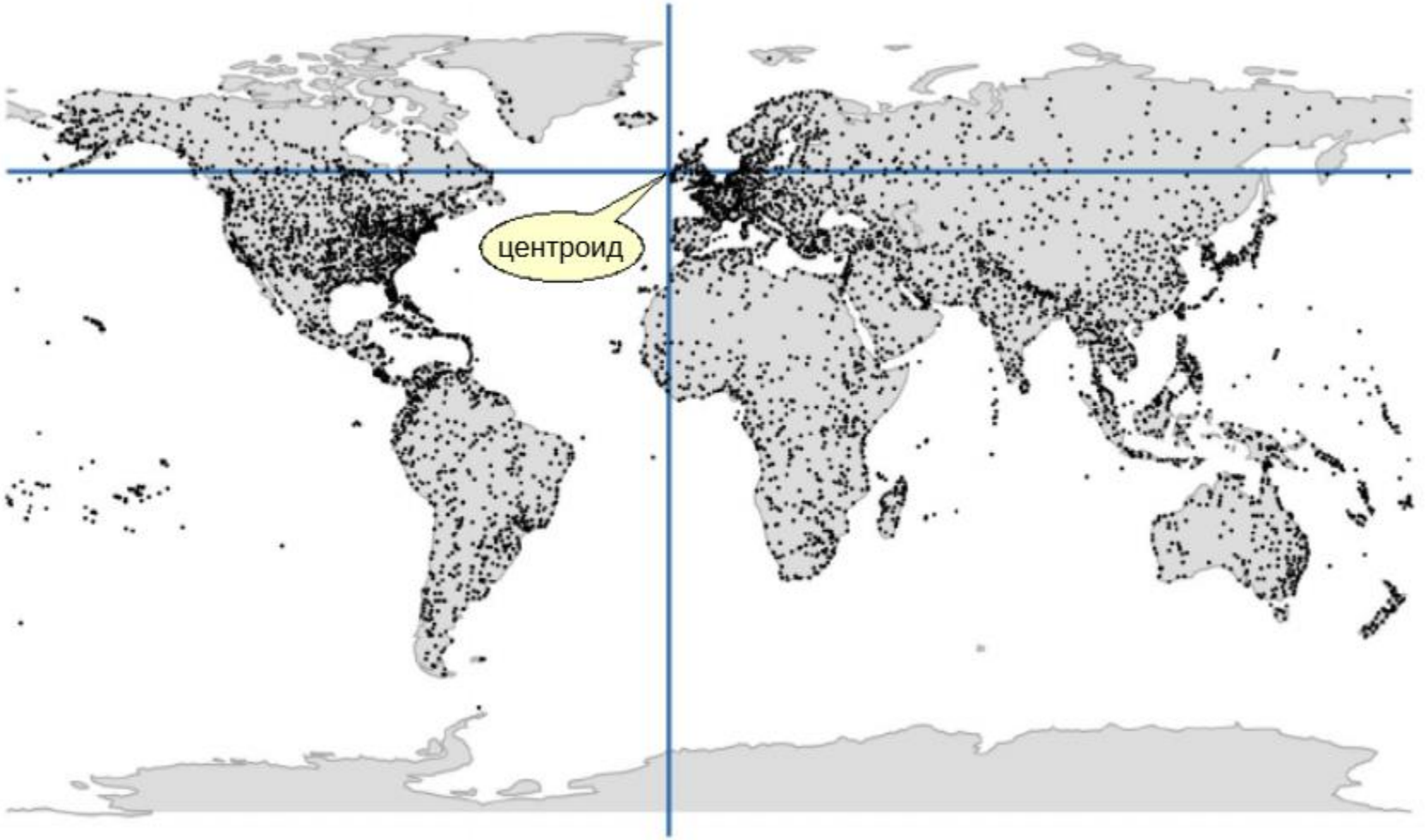
SP-GiST индекс

- SP-GiST расшифровывается как **space partitioning GiST**. Это тоже обобщенное дерево поиска, но оно строится путем разбиения пространства поиска на непересекающиеся области.
- Подходит для деревьев, IP-адресов, телефонных номеров

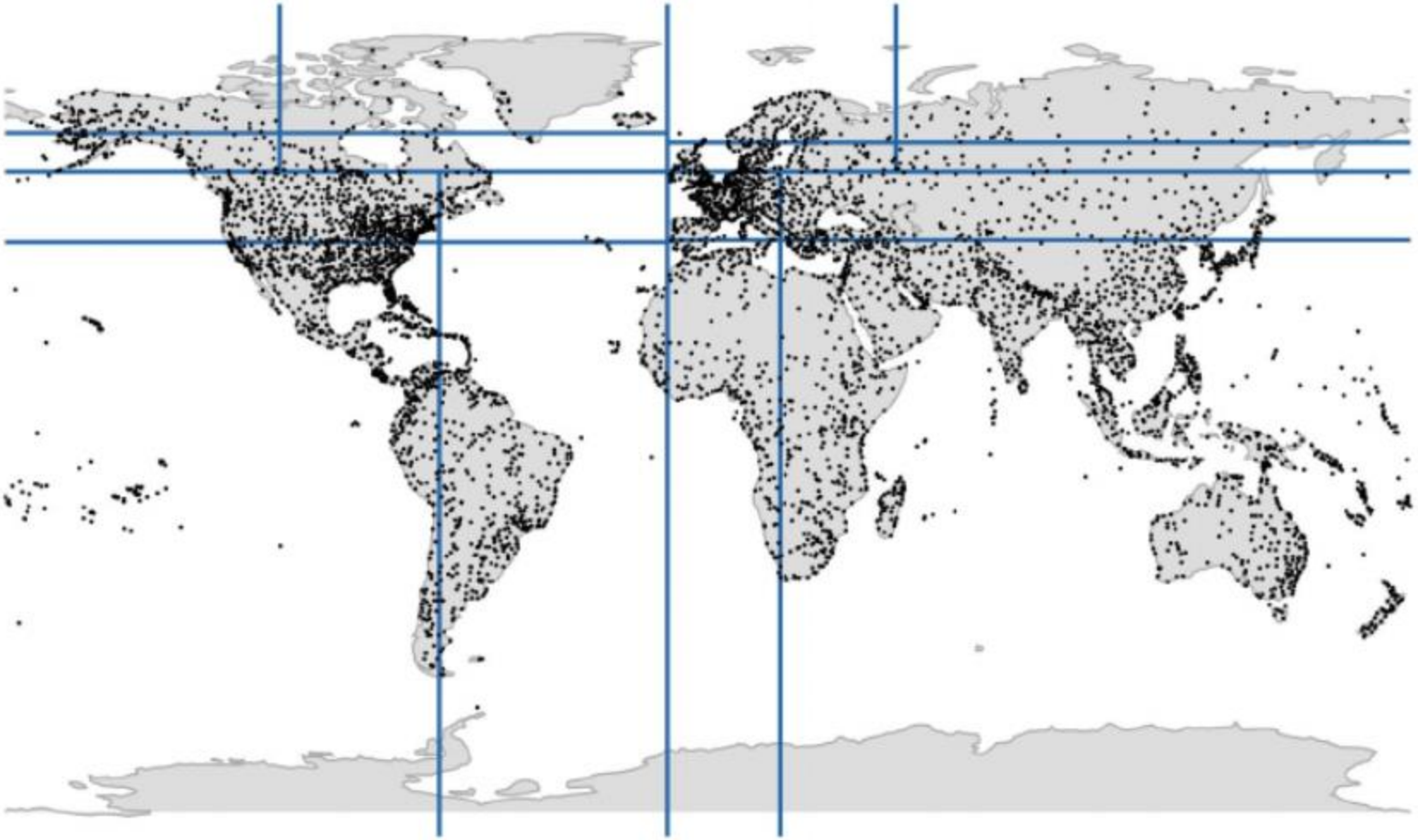
CREATE INDEX idx_spgist_name ON table USING spgist(column);

Рассмотрим пример индекса SP-GiST для точек на плоскости. Один из вариантов — дерево квадрантов. Корневой узел разбивает плоскость на четыре части (квадранта) по отношению к выбранной точке-центроиду.

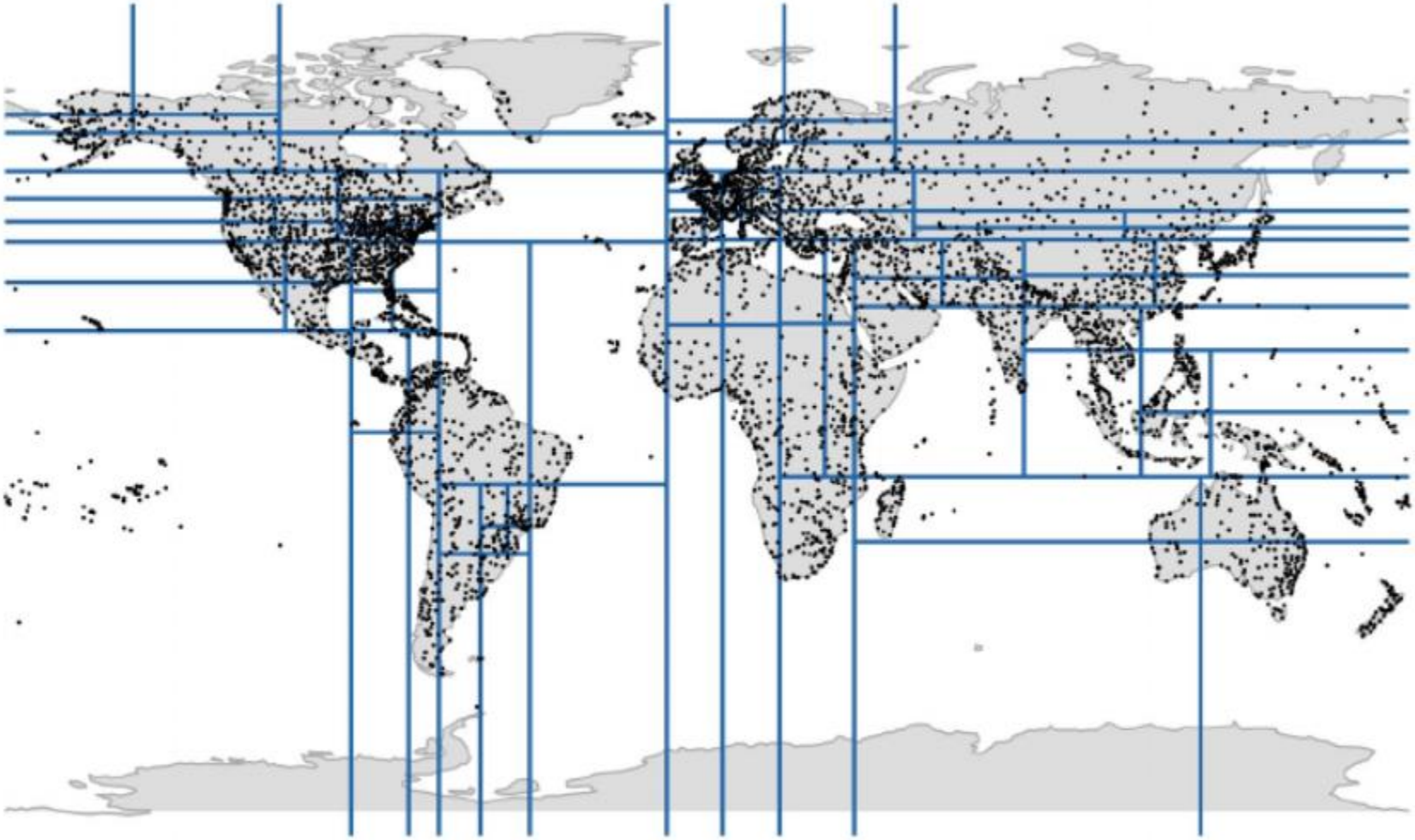
SP-GiST индекс



SP-GiST индекс



SP-GiST индекс



GIN индекс

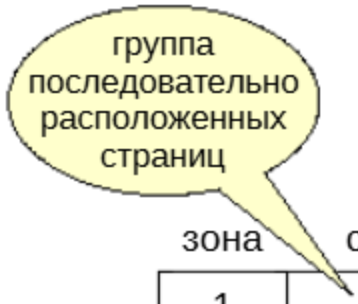
- GIN — generalized inverted index, обобщенный инвертированный индекс.
- Эффективен для массивов и полнотекстового поиск, быстро ищет наличие элементов
- Проверка соответствия документа поисковому запросу
- Проверка вхождения элемента в массив
- Поиск документов JSON по ключам или значениям

CREATE INDEX idx_gin_name ON table USING gin(column);

- Идею этого индексного метода проще всего понять на примере предметного указателя в обычной книге. На страницах книги встречаются термины, а в предметном указателе перечислены в алфавитном порядке все термины с указанием номеров страниц,на которых они присутствуют.

BRIN индекс

```
SELECT * FROM t WHERE temperature BETWEEN 20 AND 30;
```



зона	страницы	сводная информация
1	1 .. 128	{ −15 .. −3 }
2	129 .. 256	{ −7 .. −10 }
3	257 .. 384	{ 6 .. 19 }
4	385 .. 512	{ 15 .. 25 }
5	513 .. 640	{ 21 .. 32 }
6	641 .. 768	{ 13 .. 19 }
7	769 .. 896	{ 3 .. 15 }
8	897 .. 1024	{ −5 .. 6 }
9	1025 .. 1152	{ −18 .. −4 }

BRIN индекс

- Сжимает данные по диапазонам

CREATE INDEX idx_brin_name ON table USING brin(column);

- Список зон со сводной информацией зона охватывает группу последовательно расположенных страниц
- Сводная информация: минимум, максимум и т. п.
- Требуется корреляция с физическим расположением строк
- Не хранит ссылки на версии строк (только сканирование по битовой карте)
- Предназначен для очень больших таблиц (с упорядоченными данными)
- Небольшой размер
- Настраиваемое соотношение размера и точности

Индексы в PostgreSQL

Тип индекса	Лучше всего подходит для	Операторы (примеры)	Аналогия
B-tree	Уникальные значения, диапазоны, сортировка	=, >, <, >=, <=, BETWEEN, LIKE 'abc%'	Алфавитный указатель в книге
Hash	Строгое равенство	=	Быстрый поиск по ключу в словаре
GiST	Геометрия, полнотекстый поиск, диапазоны	&&, @>, <->, @@	Карта, разбитая на квадраты (с разной детализацией)
SP-GiST	Иерархические данные, IP-адреса	~, <<, >>=	Дерево телефонных кодов страны/города
GIN	Составные значения: массивы, JSON, полнотекстовый поиск	@>, @@, ?, `	Предметный указатель в конце книги (слово -> страницы)
BRIN	Огромные таблицы с физическим порядком	=, >, < (для упорядоченных данных	

Другие возможности

- Уникальные индексы (UNIQUE)
- Частичные индексы (WHERE ...)
- Многоколонные индексы
- Индексы по выражениям

Частичные индексы

- Индексы создаются только для строк, удовлетворяющих условию WHERE
- Уменьшают размер и ускоряют запросы по подмножеству данных

CREATE INDEX idx_active_users ON users(last_login)

WHERE active = true;

Многоколонные индексы

- Индекс по нескольким колонкам
- Важно учитывать порядок колонок

CREATE INDEX idx_name_age ON people (last_name, first_name, age);

Индексы по выражениям

- Индексируются результаты вычисления выражения или функции
- Полезно для запросов с вычисляемыми условиями

CREATE INDEX idx_lower_email ON users(LOWER(email));

Индексы по выражениям

- Индексируются результаты вычисления выражения или функции
- Полезно для запросов с вычисляемыми условиями

CREATE INDEX idx_lower_email ON users(LOWER(email));

Выбор индекса

EXPLAIN ANALYZE

*SELECT * FROM t WHERE t2 = ‘aaa’;*